Keimli: a decompressive cellular automaton

Jyrki Alakuijala

Draft assembled 2026-10-22 from the section files under `paper/`. Sections not yet drafted are listed at the end.

Abstract

Artificial life runs either programs, which give the richest phenomena but are general computations that can only be understood by being run, or rules, which are tractable but compute nothing. We propose a lattice whose local rule is a decompressor: each cell's 128 bits are read once as a Brotli-like stream against a shared dictionary and the decoded bytes replace the cell, so that every cell is a program that is run every turn, while the rule is a fixed-depth map, linear in the cell's size, whose image, fixed points and cycles can be sampled, enumerated and even prescribed. The decompression alone only repeats; we manage the entropy around it with two classical automata, neighbours trading bits and bit planes smoothed by majority, whose strengths vary across the image so that one frame holds the whole parameter range. The construction separates the two things a running program does, the subtle entropy reduction of its memory and the finesse of computation itself: the decoder has the first in full and the second not at all. We show that the replicators of computational life are, here, fixed points and short cycles of the rewriting map, listed by census and built by design, that their spread is the work of the medium, and that late and elaborate appearances are those of rare attractors in a slow medium rather than of computation. Twenty-five emulated processors on the same lattice form the comparison arm. The lattice, its decoders and processors, and every experiment run in the browser from a single page, which is the artifact.

1 Introduction

Artificial life has two traditions, and they have little to say to each other. One runs programs. In Core War, two warriors written in a small assembly language share a circular memory and try to overwrite each other [1]; in Tierra and Avida, self-copying programs compete for memory and processor time, and parasites, immunity and arms races arise that nobody wrote [2, 3, 4]; in the most recent work of this kind, self-replicators emerge unprompted from random populations of tiny programs [5]. The phenomena are the richest the field has. They are also the hardest to account for. A population of programs is a general computation: what a configuration will do is known only by running it, no quantity is conserved that an observer could follow, and the explanation of why a parasite appeared in one run is a story about that run. The other tradition runs rules. The Game of Life, the elementary automata, the voting rules and the reaction-diffusion systems apply the same map of a few bits to one bit everywhere [6, 7, 8]. They can be measured, perturbed, classified and in places proved [9, 10, 11], and their phenomena are those of statistical physics: domains, fronts, coarsening and phase transitions [12, 13]. In them both things that a running program does are plain: the rule's effect on entropy is simple and transparent, a few bits in and one bit out, and there is no computation at all. In a population of programs the same two things are present in full and entangled. Running a program reduces the entropy of its memory in a way that is subtler and more intricate than any Life-like rule can, the output being shaped by a library, many-to-one in a structured way, different at every position of the code; and, on top of that, the program computes, with loops, self-reference and in principle universality. Two sources of opportunity, and no way, in the first tradition, to tell which of the phenomena draws on which.

This article proposes a system between the two, and a method for reading it. The local rule is a decompressor. Each cell of a lattice holds 128 bits; once a turn, every cell's bits are read from the first to the last as the

commands of a Brotli-like stream against a shared dictionary, a header selecting a window of the dictionary and then insert-and-copy commands with distances from a short ring or given explicitly [14, 15], and the sixteen decoded bytes replace the cell. The cell is a program and the dictionary its library, and the whole lattice is a population of programs that are all run every turn. What this keeps from the first tradition is that a cell's content is code which determines its own successor through a non-trivial interpreter. What it drops is what makes computation hard. The decoder reads every input bit once and never reads its own output; it has no loop, no arithmetic on addresses and no halting question. A turn of a cell is linear in the cell's size whatever the cell holds, and the map from a cell to its successor is a fixed-depth function of the dictionary that can be sampled, enumerated and inverted. We will show that for the dictionaries used here the output of that map over random inputs carries less than 94 bits of entropy, that it has no fixed points and a small set of short cycles onto which every cell falls within a few tens of turns, that shuffling the dictionary's bytes changes the cycles in a way the census of attractors reads off, and that the cycles can be prescribed: by placing in the dictionary the words that a cell's own head points at, one builds dictionaries whose cells walk Gray-coded cycles of eight states two bits apart, for as many types as the dictionary has room for (Section 4, Figure 2).

The choice of a decompressor follows an analogy that the measurement of complexity already lives by. The complexity of a genome, a recording or a text is measured in practice by Lempel-Ziv complexity and by compression distances [16, 17, 18], not by Kolmogorov complexity, because the latter is uncomputable and the former is a linear-time, program-free proxy good enough for the purpose [19, 20]. A decompressor is the operational content of that proxy, the part of running a program that can be done in a single pass over the input. We put it inside the dynamics for the reason the measurement community put it inside the measure.

A decompressor on its own only repeats. A cell decodes, the output decodes again, and after a short transient the cell is on a cycle of the dictionary's map; a lattice of such cells is a field of independent clocks, and nothing spatial happens. The method we propose is to manage the entropy around that repetition from outside, with two stages that are each a classical automaton. A mixing stage lets neighbours trade bits: wherever two adjacent cells differ, the bits are exchanged with a probability p, the uniform crossover of genetic algorithms [21, 22] applied on a lattice without a fitness [23], which raises a cell's entropy by giving it its neighbours' bits. A settling stage smooths every bit plane with a majority filter, a voting rule in Vichniac's sense [8], which lowers the entropy by making neighbours agree. The decoder runs inside the entropy budget that the two stages set, and the budget is a parameter of the experiment, not of the rule.

Both sides of the budget are varied across the image. The trading probability runs as a gradient across the columns and the smoothing probability as a gradient down the rows, so that one lattice holds the whole two-parameter range at once, entropy supply along one axis and entropy removal along the other. One frame then shows the frozen, the ordered and the noisy regime and the fronts between them, and a dictionary can be read in all its regimes without a parameter sweep: the frame is the sweep (Figure 1). The same frame, with one point's parameters everywhere, is the control, and the uniform-parameter runs of Section 5 are that control done systematically. Varying the budget in space rather than in time also lets one judge the full range of a rule set after a few adjustments, rather than after a search over rules, which is how the dictionaries of this article were found and, later, built.

The three-stage construction makes a kind of experiment possible that neither tradition offers: ablation. Each stage can be switched off and each pair run without the third. The first thing this did was to correct our own first hypothesis. We expected the decoder to be what stops the mixing and settling pair from coarsening into one domain. It is not: the pair freezes on its own under the deterministic hashing that decides the trades and the votes, and coarsens as a power law when that hashing is annealed, so the finite domain size of the full system is pinning by a weak quenched disorder [24], and what the decoder contributes is the oscillation of the cells on their cycle and the transient that returns a perturbed cell to it (Section 5). That a hypothesis could be put and refuted in a system of programs is, to our knowledge, new.

The purpose of all this is to separate the two sources. A decompressor has the first in full: its effect on a cell's entropy is as subtle as a program's, shaped by the dictionary, many-to-one in a structured way, unequal across the positions of the cell, and it lowers the entropy of its own memory while, in a mixing medium, raising it around, a local entropy pump. It has the second not at all: it cannot loop, refer to itself or compute. Whatever the decompressive automaton reproduces of the phenomena of Core War and Tierra therefore draws on the entropy reduction of running a program and not on computation; whatever it fails to reproduce under any entropy budget is a candidate for the finesse of computation itself. To make the comparison, the same lattice runs emulated processors as its rewriting rule, the Z80, 6502, PDP-11, 68000, VAX and Redcode among twenty-five, on single cells or as persistent machines on every link between neighbours, under the same mixing, settling and gradients (Section 6). Two things the first tradition has are absent here on purpose: nothing replicates, a cell is rewritten and never copied, and nothing is selected, no score is kept. A phenomenon that appears without either is one that replication and selection did not cause.

The contributions are these. (i) A lattice whose update rule is a decompressor, which we believe has not been proposed before; the nearest prior work compresses with automata [25, 26] or measures automata by compression [17, 27], and the closest construction, a bit string read as codons modulo a grammar [28, 29], is a genotype-to-phenotype map applied once per individual, not an iterated spatial rule. (ii) An analysis of the decoder as a map: its output entropy, its attractor census, the dependence of the cycles on the dictionary's byte order, and dictionaries with a prescribed attractor structure. (iii) The managed entropy budget, varied across the image, as the method that turns a field of clocks into a spatial system, with the one-frame phase diagram and its uniform controls. (iv) The ablation of the three stages, the retraction of the coarsening hypothesis and the identification of hash pinning. (v) The comparison between the decoder and the processors on the same lattice: cycles and types, invasion with a rule for which type wins, phase synchronisation across domains, coexistence and wetting layers, and coarsening under a neighbourhood vote, as phenomena of the second kind; and the list of what needs a processor.

Section 2 defines the lattice, the three stages and the decoder to the bit, and the deterministic hashing that makes the whole a cellular automaton. Section 3 relates the construction to the two traditions and to the compression and edge-of-chaos literatures. Section 4 analyses the decoder as a map and presents the designed dictionaries. Section 5 reports the ablations, the uniform-parameter scans and the one-frame diagram. Section 6 runs the processors and draws the comparison. Section 7 discusses what the separation of the two kinds of phenomenon does and does not establish, and the limits of a model without replication. The lattice, the decoders, the processors and every experiment of the article run in the browser from a single page, which is the artifact.

2 Methods

The lattice is a plane of 128-bit cells with edges, not a torus, 512 × 512 cells in the figures unless stated. A turn applies three stages to every cell, in this order, and the view is drawn once per turn.

Mixing. Each cell exchanges bits with each of its four neighbours once a turn, in four phases, left-right and up-down at even and odd offsets, the pattern turned by ninety degrees each turn so that no direction acts last. At every position where the two cells differ, the bits are exchanged with probability p, a uniform crossover; the exchanges of a turn are decided on a snapshot of the lattice taken before the first phase. p is a parameter of the cell's column, a gradient from the left edge's value to the right edge's, mapped to the rate $2p(1 - p)$ so that the identity sits at both ends and the strongest mixing in the middle. Exchanges conserve the number of set bits. An alternative, synchronous rule, in which every bit where a neighbour differs is taken from one neighbour and written to the cell alone, is available for comparison and is isotropic by construction.

Settling. Each of the 128 bit planes is smoothed by a majority filter over the cell and its neighbourhood, the 3 × 3 block by default, computed synchronously from the previous state. A bit that disagrees with the majority is

replaced with probability p, a parameter of the cell's row, a gradient from the top edge's value to the bottom edge's. The pass is applied one to three times a turn.

Rewriting. The cell's 128 bits are read as a coded stream and the decoded 16 bytes replace them. The reference decoder is Brotli-like [14]: after six skipped bits, a header of $\lceil \log_2 V \rceil$ bits selects a window of V positions of a shared dictionary, and the stream then consists of insert-and-copy commands, each a literal count, a copy length, a distance taken from a ring of the last distances or given in seven bits, and, for copies from the dictionary, one of Brotli's four word transforms. The input wraps round the cell, consumption is capped, and whatever the stream does not produce is left as it was. Two cells can be joined into one 32-byte stream before decoding and split after, concatenated or interleaved by bit, bit pair, nibble or byte, with the two roles alternating by turn; a cell can also be joined with the bit-plane vote of its four neighbours instead of a neighbour, and only the cell written back. Other decoders are available as rewriting rules, among them a plain LZ copier, two codebooks made for cycle dynamics, and byte-format decoders from Morse to LZ4; and so are twenty-five emulated processors, from the Z80 and 6502 through the PDP-11, 68000, VAX and RISC-V to Redcode, each running a fixed number of instructions on the cell's bytes as its memory or, on cell pairs, as a persistent machine on every link between neighbours that advances one instruction a round and reads the round's snapshot. Transforms (inversion, reversal, the DCT, two sorts and an arithmetic drift) serve as controls.

Hashing. No stage draws a random number. A trade or a vote is decided by hashing the 32 bits that surround the position in the two cells, or in the one cell, and comparing the hash with the threshold that p encodes; the hash is a fixed finaliser with a salt of zero. The same bits therefore decide the same way at every place and every turn, and the whole is a deterministic cellular automaton whose randomness is quenched in its configuration; p is the fraction of contexts that decide to act, not a probability drawn each time. For control, the salt can be taken from the turn number, from the cell's position, or both, which breaks the invariance in time or in space and gives annealed noise; the article says for every run which it used.

Observables. The page reports per turn the bits exchanged by mixing, changed by settling and changed by rewriting, the byte histogram and its entropy, and the fraction of distinct cells. The scripts under `paper/experiments` add the spatial correlation of cell contents along each axis and the correlation length, the share of cells on a known cycle and their phase coherence, the number and size of domains, and the attractor census of a decoder over random inputs. Every number in the article comes from a script run on the same module the page uses, never from a screenshot.

Reproducibility. The page, its module, the dictionaries and the scripts are at [repository]; each figure names its settings line, a single string that the page accepts and that restores every control, so that any figure can be re-run in the browser. The designed dictionaries are produced by the scripts under `paper/experiments/design-*.mjs` and reproduced byte for byte by them. Exact definitions of every stage, of the decoder to the bit, of the processors and of the band-parallel execution that makes the browser version bit-identical to the single-threaded one are in the repository's documentation (`docs/lattice.md`, `docs/changers.md`, `docs/cpus.md`), which the tests under `test/` check against the module.

5.x Replicators are fixed points and short cycles

The organisms of computational life are called replicators because a program, run, leaves a copy of itself in memory. Seen as a dynamical system the statement is simpler. The interpreter is a map from the content of a memory to its content after a run; a program that recreates itself is a fixed point of that map, and a program that recreates itself after k runs, passing through $k - 1$ intermediate contents, is a point on a k-cycle. Whatever else a replicator is, it is first an attractor of the rewriting map, and the question of where replicators come from splits into two: which attractors the map has, and how a population reaches them.

The decompressive automaton answers the first question by census. The rewriting map of a dictionary is a function on 2^128 cells that can be sampled: a million random cells are decoded until each has returned to a content already seen, and the cycles are counted (Section 4). Under the reference dictionary every random cell reaches one 3-cycle within 35 turns, and there is no fixed point; each of the searched dictionaries gives one dominant short cycle; natural texts give long cycles and many competing attractors; the codebook decoders, made for the purpose, give many short attractors with balanced basins; and the designed dictionaries give exactly the 8-cycles they were built to give, three to thirty-two of them, with consecutive states two bits apart and a transient of about five turns (Table 3). In this system, then, the replicators are known before the lattice runs, their number and length are properties of the dictionary that a shuffle of its bytes changes, and they can be prescribed. They are not born; they are listed.

The second question, how a population reaches them, is where the lattice comes in, and the answer separates two things that computational life shows together. A cell on a cycle recreates itself in place; it does not spread. Spreading is the work of the medium: the mixing stage carries a cycle state's bits into its neighbours, the settling stage makes neighbours agree, and the decoder's funnel, the transient that returns a perturbed cell to the cycle, turns an approximate copy into an exact one. Under this budget a cycle takes over a domain, a domain's phase synchronises, two cycles compete at a front with a rule for which wins (the one whose head the decoder reads in fewer bits), and types coexist in a mosaic or one wets the boundary between two others (Section 5.y). None of this involves an instruction that copies; the copying is done by exchange and majority, and the program's only contribution is to be an attractor with a steep enough funnel. The dictionaries whose cycles are the same but whose funnel is shallow never take hold (Section 4), which is the same statement from the other side: a replicator that is an attractor of the map but not of its neighbourhood does not replicate.

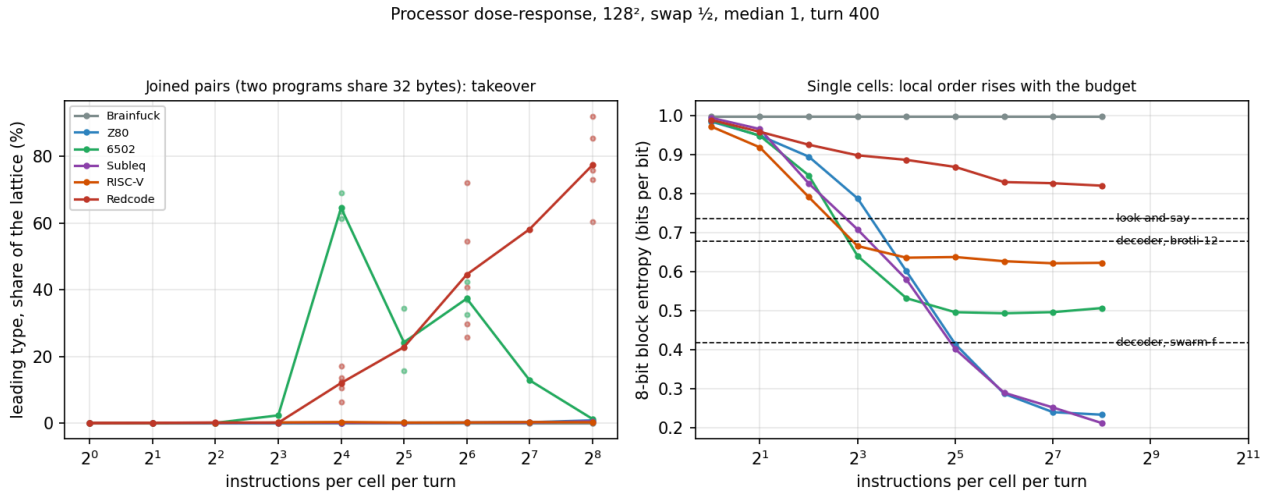
Two corollaries follow, both testable on the processor arm. First, a processor's rewriting map has attractors too, and they can be counted the same way: a cell run as a Z80, a 68000 or a Redcode warrior for a fixed number of instructions is a map on cells, and its fixed points and short cycles are its replicators in the same sense (*to show*: the attractor census for the processors, which the existing tools apply to any changer; the expectation from the decoder is many fixed points, since most random code halts or loops on itself, and few cycles). The self-replicators that emerge in random populations of small programs [5] should appear in that census as the attractors of the pairwise map, and the spread they show in a population as the work of whatever medium the population has, which there is the random pairing of programs. Second, what a processor adds over a decoder is not replication but the dependence of the attractor on the run: a processor can reach different fixed points from the same memory under different numbers of instructions and can carry state in registers the memory does not show, which the link machines of Section 6 do, and the phenomena that need this, if any, are the ones to attribute to computation proper (*to show*: the comparison table).

A third point concerns time. In the computational systems a replicator that appears late, or that is elaborate when it appears, is often read as evidence that something was computed on the way to it: that the population searched, learned or built. The decompressive automaton shows late and elaborate appearances without any such thing, and a Monte Carlo over the dictionaries says what they are (`results/replicators.md`). First appearance is never late for an attractor that matters: every type that ever held one per cent of the cells was present at the earliest turn the detector allows, in every run, because the first of N cells to fall into a basin b is expected after 1 / (N b) turns and no detectable basin is below 1 / N. What is late is takeover. A type present from the start at a share below notice takes the lattice at the pace of a front, half of a 256^2 lattice after a thousand turns for the fixed point of one searched dictionary under the page frame, about 1.5 times longer than at 128^2; and which type that is depends on the entropy budget rather than on the basin: under strong uniform settling the attractor with the largest basin wins within a few tens of turns, under the frame a fixed point whose basin is a three-hundredth of the leading cycle's wins instead, because the majority filter propagates what makes neighbours agree. The sharpest-looking structures of the page, the type mosaics of the designed dictionaries, are those that never finish taking over at all. Neither the

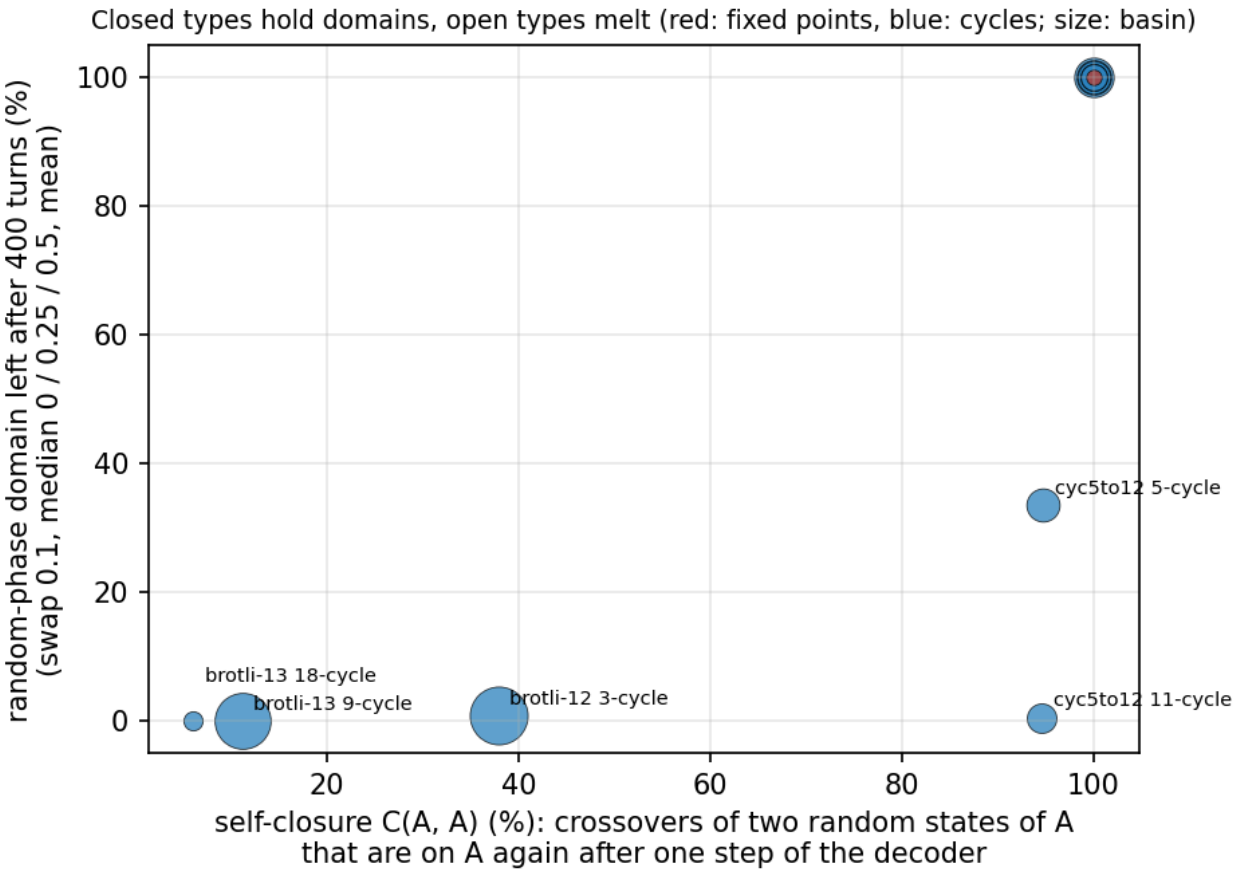
lateness nor the elaboration is the mark of a computation; both are the marks of a slow medium choosing among attractors that were all there from the start, and the decompressive automaton, which cannot compute, produces them on schedule (*to show*: the long runs for any type whose basin is below 1 / N, which would be the only genuine late appearances, and the takeover time against the lattice size and the budget as the curves a computational system's late replicators should be compared with before anything is attributed to the computation).

The statement to carry into the discussion is therefore this. In the decompressive automaton, replication decomposes into an attractor of the rewriting map, which is a matter of enumeration, and a spread through an entropy budget, which is a matter of statistical physics; nothing in it is a replicator in the sense of a program that copies itself, and yet the lattice shows the type, the invasions and the coexistence that replicators are credited with. The burden is on the processor arm to show what is left when those two are taken away.

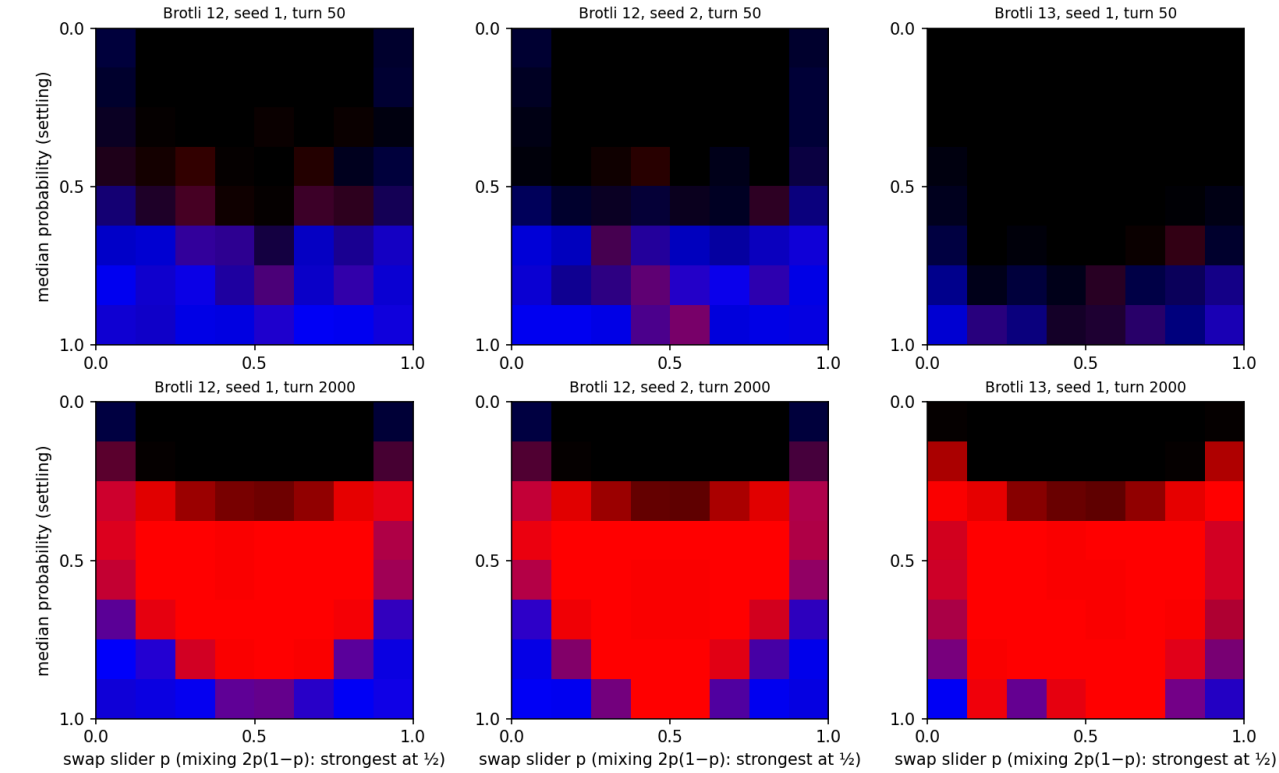
Figures



dose-response.png



mechanism-closure.png



mechanism-frame.png

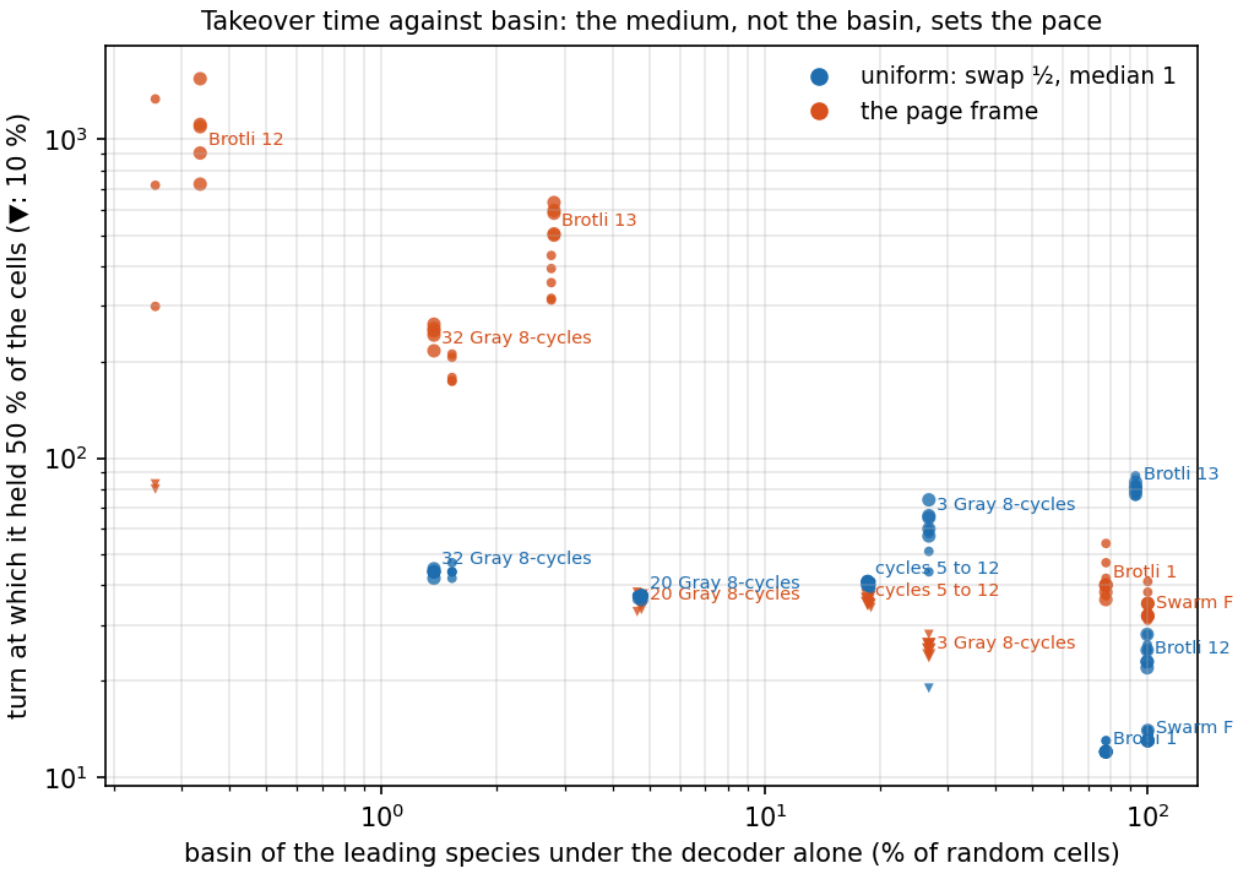


Figure R2. The turn at which the leading type held half the cells against its basin under the decoder alone, per seed. Under the uniform setting the takeover takes 24 to 80 turns whatever the basin; under the frame the time falls with the basin from about a thousand turns at 0.3 % to forty at 100 %, and the winner can be a fixed point whose basin is a three-hundredth of the leading cycle's.

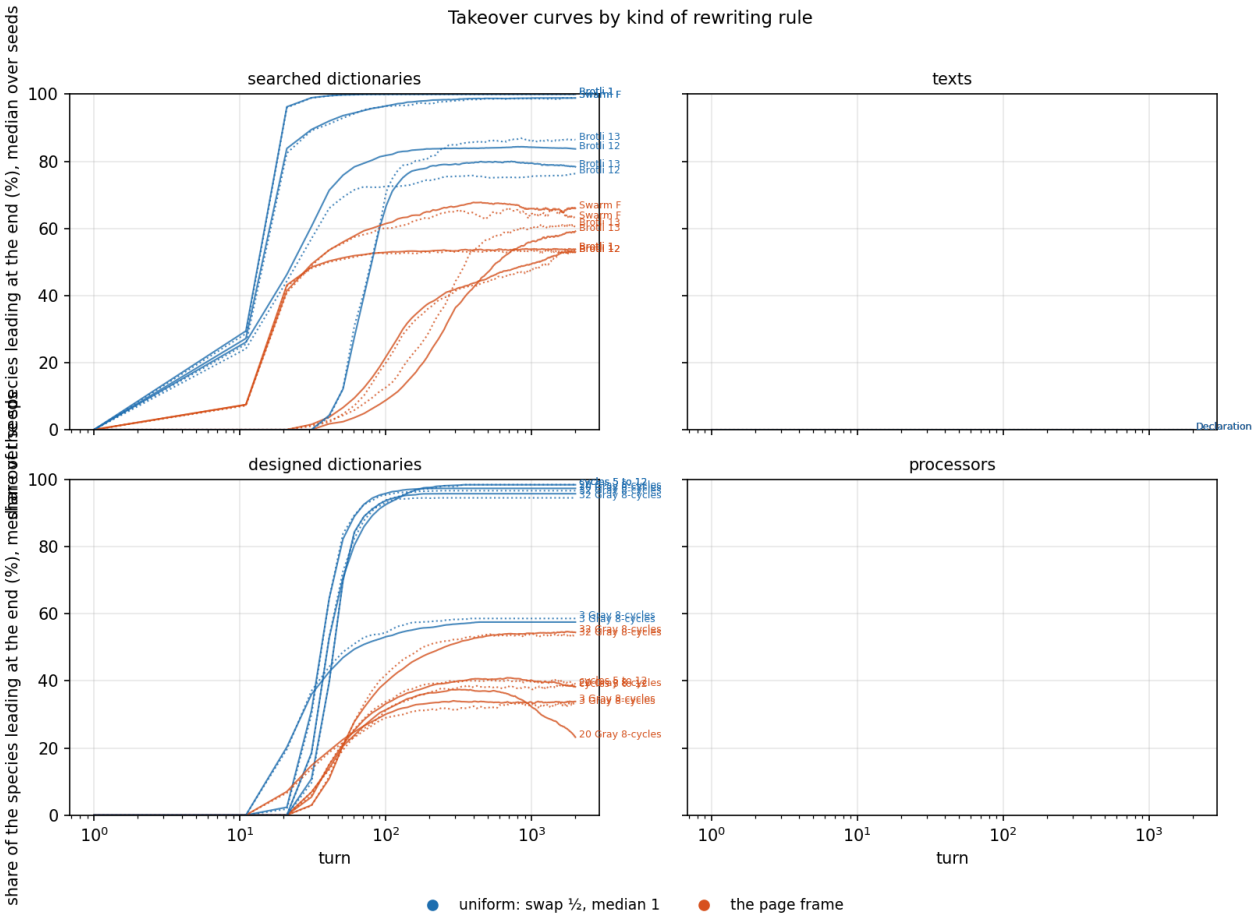


Figure R3. Takeover curves by kind of rewriting rule: the share of the type leading at the end, median over seeds, one curve per dictionary or processor; searched dictionaries, texts, designed dictionaries and processors in the four panels.

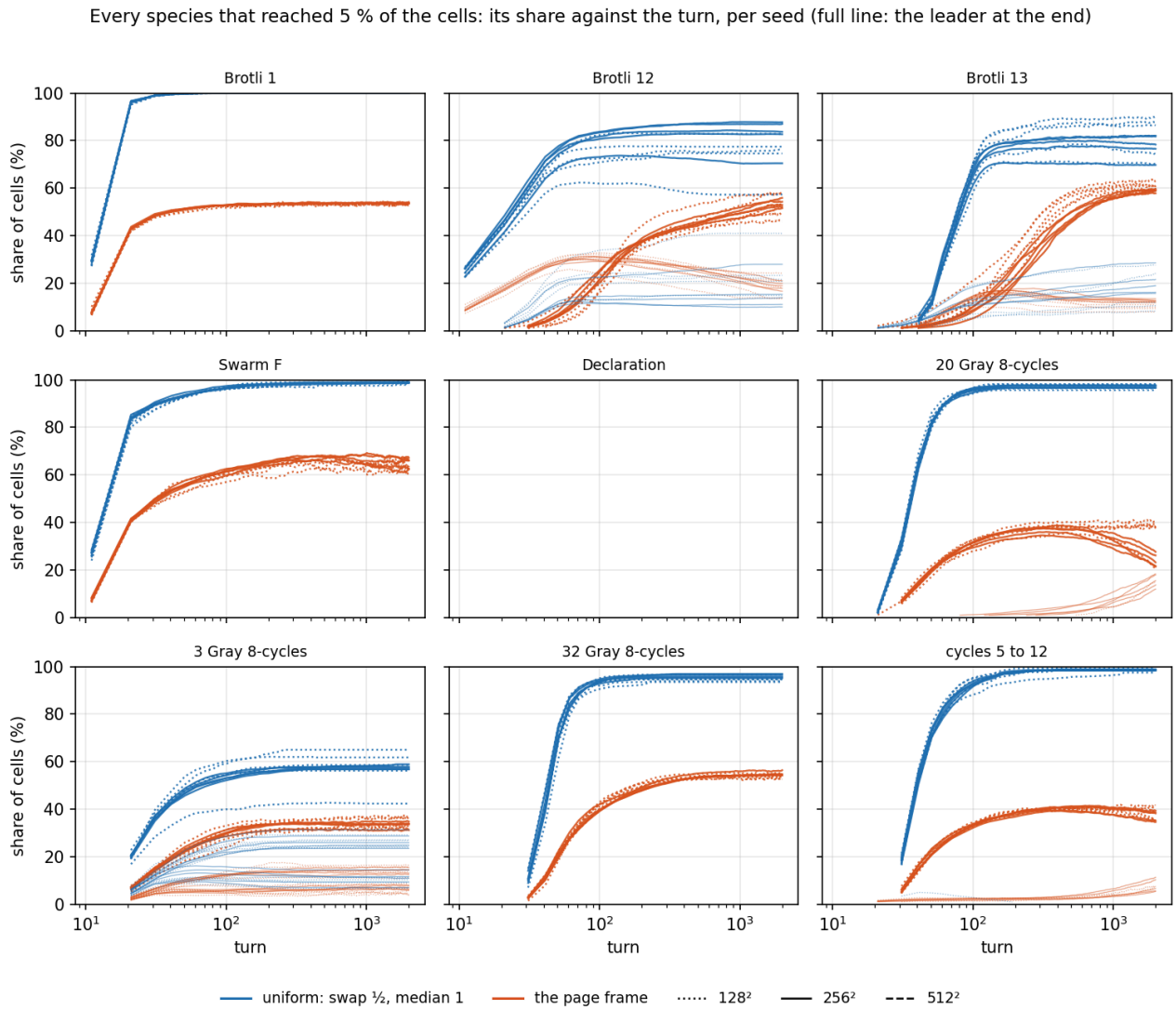


Figure R1. Takeover by the leading type: the share of cells on the most common cycle against the turn, one panel per dictionary, every seed a line; blue the uniform setting (swap ½, median 1), orange the page frame; dotted 128^2, solid 256^2. Every type that ever mattered is present as soon as its period can be seen (a k-cycle from turn $2k - 1$); what differs is how fast, and whether, it takes the lattice. The type leading at the end is the full line, the others thin: under the frame the large-basin cycle of Brotli 12 and 13 rises first and gives way to a fixed point.

Sections still to draft

3 Related work (the map is in `RELATED_WORK.md`); 4 The decoder as a map and the designed dictionaries (`results/decoder-map.md`, `results/design-gray3.md`); 5 Ablations, scans and the one-frame diagram (`results/ablation-*.md`, `results/swap-order.md`, `results/life-vote.md`); 5.y Type, invasion and coexistence (`results/design-gray3.md`); 6 The processors and the comparison; 7 Discussion. The replicator Monte Carlo (`experiments/replicators.mjs`) is running and will give Section 5.x its table.

References

1. Dewdney, A. K. “Computer recreations: in the game called Core War hostile programs engage in a battle of bits”. Scientific American. 250, 14–22. 1984

2. Ray, Thomas S. “An approach to the synthesis of life”. Artificial Life II. X, 371–408. 1991
3. Adami, Christoph, Brown, C. Titus. “Evolutionary learning in the 2D artificial life system “Avida””. Artificial Life IV. , 377–381. 1994
4. Lenski, Richard E., Ofria, Charles, Pennock, Robert T., Adami, Christoph. “The evolutionary origin of complex features”. Nature. 423, 139–144. 2003
5. Agüera y Arcas, Blaise, Alakuijala, Jyrki, Evans, James, Laurie, Ben, Mordvintsev, Alexander, Niklasson, Eyvind, Randazzo, Ettore, Versari, Luca. “Computational life: how well-formed, self-replicating programs emerge from simple interaction”. arXiv:2406.19108. 2024
6. Wolfram, Stephen. “Universality and complexity in cellular automata”. Physica D. 10, 1–35. 1984
7. Langton, Christopher G. “Computation at the edge of chaos: phase transitions and emergent computation”. Physica D. 42, 12–37. 1990
8. Vichniac, Gérard Y. “Simulating physics with cellular automata”. Physica D. 10, 96–116. 1984
9. Mitchell, Melanie, Hraber, Peter T., Crutchfield, James P. “Revisiting the edge of chaos: evolving cellular automata to perform computations”. Complex Systems. 7, 89–130. 1993
10. Crutchfield, James P., Hanson, James E. “Turbulent pattern bases for cellular automata”. Physica D. 69, 279–301. 1993
11. Shalizi, Cosma Rohilla, Haslinger, Robert, Rouquier, Jean-Baptiste, Klinkner, Kristina Lisa, Moore, Cristopher. “Automatic filters for the detection of coherent structure in spatiotemporal systems”. Physical Review E. 73, 036104. 2006
12. Bray, Alan J. “Theory of phase-ordering kinetics”. Advances in Physics. 43, 357–459. 1994
13. de Oliveira, Mário J. “Isotropic majority-vote model on a square lattice”. Journal of Statistical Physics. 66, 273–281. 1992
14. Alakuijala, Jyrki, Szabadka, Zoltán. “Brotli compressed data format”. 2016
15. Alakuijala, Jyrki, Farruggia, Andrea, Ferragina, Paolo, Kliuchnikov, Eugene, Obryk, Robert, Szabadka, Zoltán, Vandevenne, Lode. “Brotli: a general-purpose data compressor”. ACM Transactions on Information Systems. 37, 4:1–4:30. 2019
16. Ziv, Jacob, Lempel, Abraham. “A universal algorithm for sequential data compression”. IEEE Transactions on Information Theory. 23, 337–343. 1977
17. Kaspar, F., Schuster, H. G. “Easily calculable measure for the complexity of spatiotemporal patterns”. Physical Review A. 36, 842–848. 1987
18. Cilibrasi, Rudi, Vitányi, Paul M. B. “Clustering by compression”. IEEE Transactions on Information Theory. 51, 1523–1545. 2005
19. Zenil, Hector, Soler-Toscano, Fernando, Delahaye, Jean-Paul, Gauvrit, Nicolas. “Two-dimensional Kolmogorov complexity and an empirical validation of the coding theorem method by compressibility”. PeerJ Computer Science. 1, e23. 2015
20. Chaitin, Gregory. “Proving Darwin: Making Biology Mathematical”. Pantheon Books. 2012
21. Syswerda, Gilbert. “Uniform crossover in genetic algorithms”. Proceedings of the Third International Conference on Genetic Algorithms. , 2–9. 1989
22. Spears, William M., De Jong, Kenneth A. “On the virtues of parameterized uniform crossover”. Proceedings of the Fourth International Conference on Genetic Algorithms. , 230–236. 1991
23. Alba, Enrique, Dorronsoro, Bernabé. “Cellular Genetic Algorithms”. Springer. 42. 2008
24. Sicilia, Alberto, Arenzon, Jeferson J., Bray, Alan J., Cugliandolo, Leticia F. “Geometric properties of two-dimensional coarsening with weak disorder”. EPL (Europhysics Letters). 82, 10001. 2008

25. Haukeli, Martin. “Lossless data compression using cellular automata”. 2012
26. Macura, Wiktor. “Data compression with cellular automata”. Presentation, NKS 2006 Wolfram Science Conference, Washington, DC. 2006
27. Cisneros, Hugo, Sivic, Josef, Mikolov, Tomas. “Evolving structures in complex systems”. 2019 IEEE Symposium Series on Computational Intelligence (SSCI). , 230–237. 2019
28. Ryan, Conor, Collins, J. J., O'Neill, Michael. “Grammatical evolution: evolving programs for an arbitrary language”. Genetic Programming: First European Workshop (EuroGP'98). 1391, 83–96. 1998
29. O'Neill, Michael, Ryan, Conor. “Grammatical evolution”. IEEE Transactions on Evolutionary Computation. 5, 349–358. 2001